

What Tatami Correctness

One theorem in five parts, and the 162 results they rest on. Every statement below is checked by Lean's kernel; none is assumed, and none is left open.

Checked by Lean 4.33.0, no Mathlib Open goals none Theorems 163
docs/phase_1/guarantees.html ·rebuild with make -C docs guarantees

01

What “proved” means here

Every claim in this document is a theorem in the repository, accepted by Lean's type checker. Nothing is asserted in prose and taken on trust.

No open goals

Lean records an unfinished proof as `sorry` and warns on every one. The build reports none. This is mechanical, not a claim about our diligence: `lake build` prints a warning for each, and there are no warnings to print.

No extra axioms

`#print axioms pipeline_correct` reports `propext`, `Classical.choice` and `Quot.sound` — the three Lean itself is built on. In particular *not* `sorryAx`, which is what an unfinished proof would leave behind even if the warning were suppressed.

Mathlib is not used. Everything below rests on Lean's own library, which means the lemmas about lists, orders and sorting that the proofs need were either found in core or proved here.

02

The guarantee

For a corpus inference accepted and a schema the generator accepted, five things hold of the output. This is one theorem, `pipeline_correct`; everything else in this document is what it rests on.

#	GUARANTEE	GROUPS
1	Well-formedness. The emitted signature repeats no module name, no value name within a module, and no field name within a record. That is what “a legal OCaml signature” amounts to for this fragment. Without it the generator could emit two record fields of one name, which OCaml rejects outright.	gen_wellFormed
2	Canonicity. The schema is determined by the corpus as a collection of documents, not by the order they arrived in. Shuffle the corpus and the answer is <i>equal</i>, not merely equivalent. Without it two runs over the same data could disagree, and the output would depend on the order files happened to be read in.	infer_perm

- 3 **Structure preservation.** The tree the documents induce and the graph a reader finds in the generated signatures are the same graph — every edge one has the other has, distinct tables get distinct modules, and every table is reachable from the root.
- Each clause earns its place. Without the first two an edge could be lost; without the next two one could be invented; without injectivity two tables could collapse into one module and the agreement would be vacuous; without rootedness a table could be declared and never reached, which is a table the loader creates and never fills.
- 4 **Principality.** Each member is given the principal type of the values seen there: one that admits every one of them (*adequacy*), and the least type that does (*minimality*).
- Adequacy alone is nearly free — a corpus of integers is adequately described by `float`, or by `int option`. Minimality rules the loose answers out, and the two together pin the type from both sides.
- 5 **Nullability.** A column is optional exactly when some document lacked a value there. Two halves: every visit to a table is accounted for at each member — a value, an explicit `null`, or an absence — and the `option` flag is set precisely when the member carried a value in fewer than all of them.
- Absence is computed by *truncating* subtraction. Without the counts identity a member some document had omitted could report zero absences, come out non-optional, and the generator would emit `string` where the data needs `string option`. That output compiles and then fails on the first document missing the member, with nothing to say what went wrong.

`gen tree complete``gen tree sound``gen moduleName injOn``gen rooted``infer admits` `infer least``infer counts`

What it does not say. Every part is conditional on both stages returning `ok`. Both refuse inputs — a member holding two types with no common type, a schema whose tree a signature cannot express — and the guarantee is about what is emitted, not about what is accepted. It also says nothing about rows: the program emits a description of tables, and the loader that fills them is OCaml, outside what Lean sees here.

03

What rests on what

The dependency tree, read off the proof terms themselves rather than written by hand: a result appears under another exactly when the second proof mentions the first. A result used in more than one place is expanded the first time it appears and marked thereafter.

1. Well-formedness

`signature_wellFormed` `Correctness` The emitted signature repeats no module name, no value name within a module and no field name within a record.

└ `gen_wellFormed` `Wellformed` Every generated signature is well formed: no module, value or record field name repeats.

└ └ `certify_wellFormed` `Wellformed` The generator checks its own output before returning it.

└ └ └ `okB_sound` `Wellformed` That check means what the proposition says.

└ └ └ └ `nodupNames_sound` `Wellformed` And so does its distinctness test.

2. Canonicity

`schema_canonical` `Correctness` The schema is determined by the corpus as a collection of documents, not by the order they arrived in -- shuffle it and the answer is equal, not merely equivalent.

`infer_perm` `Inference` The schema is a property of the corpus and not of the order its documents arrived in: shuffle the corpus and you get the same answer.

`Tables.merge_ok` `Merge` Merging preserves the order invariant.

`TableObs.merge_ok` `Merge` The same, one level down.

`Obs.merge_ok` `Merge` And one level down again.

`tyOrder` `Order` So are types.

`unionBy_sorted` `Sorted` And preserves order.

`belowK_of_lt` `Sorted` Transitivity carries `below` down from a smaller key.

`unionBy_belowK` `Sorted` A key below both sets is below their union.

`Tables.merge_right_comm` `Merge` Together: two documents merged into an accumulator commute, which is what the corpus fold needs.

`Tables.merge_assoc` `Merge` And grouping three of them either way gives the same answer.

`TableObs.merge_assoc` `Merge` The same, one level down.

`Obs.merge_assoc` `Merge` And one level down again.

`tyOrder` `Order` *shown above*

`unionBy_assoc` `Sorted` And associative.

`sortedK_ext` `Sorted` Two sorted sets with the same members are the same list.

`belowK_of_lt` `Sorted` *shown above*

`memBy_of_below` `Sorted` A key below everything in a set is not in it.

`unionBy_mem` `Sorted` A union is exactly the two memberships, disjointed.

`belowK_of_lt` `Sorted` *shown above*

`memBy_of_below` `Sorted` *shown above*

`unionBy_sorted` `Sorted` *shown above*

`Obs.merge_ok` `Merge` *shown above*

`mergeBy_assocOn` `Sorted` Merging is associative, for the same reason.

`lookupBy_allV` `Sorted` A value looked up satisfies whatever the list's values do.

`mergeBy_lookup` `Sorted` A merge is exactly its two lookups combined -- this is the whole content of merging, and both laws below read it off.

`below_of_lt` `Sorted` The same, for association lists.

`lookupBy_of_below` `Sorted` A key below everything in a list is not in it.

`mergeBy_sorted` `Sorted` Merging two ordered lists gives an ordered list.

`below_of_lt` `Sorted` *shown above*

`mergeBy_below` `Sorted` A key below both lists is below their merge.

`sorted_ext` `Sorted` Two sorted lists that agree on every lookup are the same list.

`below_of_lt` `Sorted` *shown above*

`lookupBy_of_below` `Sorted` *shown above*

`strOrder` `Order` Member names are strictly totally ordered.

`TableObs.merge_ok` `Merge` *shown above*

`pathOrder` `Order` So are table paths.

`Tables.merge_comm` `Merge` Merging two documents' observations gives the same answer in either order.

TableObs.merge_comm	Merge	The same, one level down, for a single table's observations.
Obs.merge_comm	Merge	And one level down again, for a single member's: counts add, type sets union, flags disjoint.
tyOrder	Order	shown above
unionBy_comm	Sorted	The same for a set: union is commutative.
mergeBy_commOn	Sorted	Merging is commutative, because combining lookups is.
lookupBy_allV	Sorted	shown above
mergeBy_lookup	Sorted	shown above
mergeBy_sorted	Sorted	shown above
sorted_ext	Sorted	shown above
strOrder	Order	shown above
pathOrder	Order	shown above
TablesOk.nil	Merge	Observing nothing is trivially in order.
except_bind_ok	Sorted	A computation that succeeded had both halves succeed.
foldl_perm_on	Inference	The same, where the laws hold only of values satisfying an invariant.
mapM_perm	Inference	Permuted inputs give permuted outputs, when both succeed.
except_bind_ok	Sorted	shown above
mapM_ok_of_all	Inference	If every element succeeds then the whole list does.
mapM_ok_pointwise	Sorted	And every input reached some output.
except_bind_ok	Sorted	shown above
mem_of_mapM_ok	Sorted	Every output of a mapped list came from some input.
except_bind_ok	Sorted	shown above
observeDocument_ok	Walk	So one document, observed from nothing, comes out in order.
TablesOk.nil	Merge	shown above
observe_ok	Walk	Everything the inference walk records is kept in key order -- tables by path, members by name, types by their order.
Tables.upsert_ok	Merge	Recording a table keeps everything in order.
TableObsOk.empty	Merge	So is a table with no members yet.
allV_map	Sorted	And preserves a property of the values.
insertBy_allV	Sorted	And preserves a property of the values.
insertBy_sorted	Sorted	Inserting one entry keeps the list ordered.
below_of_lt	Sorted	shown above
insertBy_below	Sorted	And keeps a smaller key below it.
pathOrder	Order	shown above
sortedBy_map	Sorted	Rewriting values in place leaves the order alone.
below_map	Sorted	So it leaves `below` alone.
except_bind_ok	Sorted	shown above
keep	Walk	Each record update the walk makes leaves the member list alone.
recordMember_ok	Merge	So does recording one observation at one member.
Obs.merge_ok	Merge	shown above
Tables.upsert_ok	Merge	shown above
insertBy_allV	Sorted	shown above
insertBy_sorted	Sorted	shown above
strOrder	Order	shown above

3. Structure preservation

`structure_preserved` `Correctness` The tree the documents induce and the graph a reader finds in the generated signatures are the same graph, and it is rooted.

- `gen_moduleName_injOn` `Tree` Two different tables never get the same module name, so the file's graph is the same graph and not a collapse of it.

- `genModule_name` `Tree` A table's module is named after that table.

- `except_bind_ok` `Sorted` *shown above*

- `gen_faithful` `Tree` The generator checks its input has the shape a corpus produces, so the tree properties hold of whatever it is handed.

- `gen_schemaOk` `Tree` If generation succeeded, the schema passed that check.

- `nodupNames_sound` `Wellformed` *shown above*

- `nodupPaths_sound` `Tree` The check that no two tables share a path means what it says.

- `path_mem` `Tree` A path the check found in the schema is the path of a table of it.

- `gen_modules` `Tree` Every module in the output came from one of the three generation passes.

- `except_bind_ok` `Sorted` *shown above*

- `gen_inv` `Tree` Generation is the raw pass followed by a check that hands back its input unchanged.

- `noClash_modules` `Tree` The duplicate-name check hands back its input unchanged.

- `gen_wellFormed` `Wellformed` *shown above*

- `inj_of_nodup_map` `Tree` If a list's keys are all distinct then two elements with the same key are the same element.

- `mapM_ok_map` `Sorted` A mapped list agrees pointwise with its inputs under any function that agrees on each.

- `except_bind_ok` `Sorted` *shown above*

- `mem_orderTables_complete` `Tree` The same, said of the sort as a whole.

- `mem_orderGo_complete` `Tree` And it loses none, given that no two tables share a path.

- `inj_of_nodup_map` `Tree` *shown above*

- `gen_rooted` `Tree` Every table is reachable from the root, so walking the signatures from the root module enumerates the whole schema.

- `gen_schemaOk` `Tree` *shown above*

- `reachable_sound` `Tree` Everything the search reports is genuinely reachable from the root.

- `reachGo_sound` `Tree` So does the whole search.

- `stepReach_sound` `Tree` One round of the reachability search only ever reports tables genuinely one edge further out.

- `gen_tree_complete` `Tree` Nothing in the document's tree is lost: every nesting the documents induce is declared in the generated signatures.

- `gen_tree_complete_coll` `Tree` The half of that for collections: a table whose rows sit inside another declares the lookup that finds them.

- `genModule_lookup` `Tree` A table whose rows sit in a collection declares the lookup that finds them from the holder's key.

- `except_bind_ok` `Sorted` *shown above*

- `genModule_name` `Tree` *shown above*

- `gen_faithful` `Tree` *shown above*

- `gen_module_of` `Tree` Every table of the schema has a module in the output.

- `gen_faithful` `Tree` *shown above*

- `gen_modules` `Tree` *shown above*

- `mapM_ok_pointwise` `Sorted` *shown above*

	mem_orderTables_complete	Tree	shown above
gen_tree_complete_ref	Tree	The half for references: a member that was an object and became a table leaves an accessor holding that table's key.	
genModule_accessor	Tree	Every column of the layout becomes an accessor in the signature.	
	except_bind_ok	Sorted	shown above
genModule_layout	Tree	A module was only generated if its column layout was computed first.	
	except_bind_ok	Sorted	shown above
genModule_lookup_only	Tree	And that is the only declaration of that shape, so a reader who finds one has found a parent and not something that merely resembles one.	
	except_bind_ok	Sorted	shown above
genModule_name	Tree	shown above	
gen_faithful	Tree	shown above	
gen_moduleName_injOn	Tree	shown above	
gen_module_of	Tree	shown above	
gen_module_source	Tree	Every module carrying any declaration at all is some table's.	
	genLoader_decls	Tree	A loader has no signature at all, so it carries no edges.
		except_bind_ok	Sorted shown above
	genRegistry_decls	Tree	Neither does the registry.
	gen_modules	Tree	shown above
	mem_of_mapM_ok	Sorted	shown above
	mem_orderTables_sound	Tree	The same, said of the sort as a whole.
		mem_orderGo_sound	Tree The compile-order sort invents no tables.
	inj_of_nodup_map	Tree	shown above
keyTyOf_id_not_ref	Tree	A member called `id` that is an object is refused outright, so a reference never hides in the key.	
	find?_id_eq	Tree	With distinct member names, looking up the key finds the column that is there to be found.
		inj_of_nodup_map	Tree shown above
layoutOf_keyTy	Tree	If the layout was computed, the key's type was settled first.	
	except_bind_ok	Sorted	shown above
layoutOf_mapM	Tree	If the layout was computed, every member was considered.	
	except_bind_ok	Sorted	shown above
layoutOf_member	Tree	A column produced for a member of the table reaches the layout.	
	except_bind_ok	Sorted	shown above
	mapM_ok_pointwise	Sorted	shown above
	mapM_ok_pointwise	Sorted	shown above
	memberCol_ref	Tree	A member that is an object and is not the key becomes a column holding that object's key.
gen_tree_sound	Tree	Nothing is invented: every edge a reader finds between two tables is one the documents actually induced.	
	baseCols_ref	Tree	Of the columns a table's position adds, only the back reference is a key into another table.
		keyTyOf_not_ref	Tree A table's key is never an object.
colTyExpr_ref	Tree	Only a reference column is typed as another module's key; a collection becomes a list and a scalar a scalar.	
genModule_accessor_only	Tree	And accessors are the only declarations that take a row and return another module's key.	

- `except_bind_ok` `Sorted` *shown above*
- `genModule_lookup` `Tree` *shown above*
- `genModule_lookup_only` `Tree` *shown above*
- `genModule_name` `Tree` *shown above*
- `gen_faithful` `Tree` *shown above*
- `gen_moduleName_injOn` `Tree` *shown above*
- `gen_module_source` `Tree` *shown above*
- `inj_of_nodup_map` `Tree` *shown above*
- `mem_layoutOf` `Tree` Every member that is neither the table's key nor a collection becomes a column, under its mangled name, at the type inference gave it.
 - `except_bind_ok` `Sorted` *shown above*
 - `mem_of_mapM_ok` `Sorted` *shown above*
- `memberCol_ty` `Tree` A member column keeps the member's own type.

4. Principality

`types_principal` `Correctness` Each member is given the principal type of the values seen there: one that admits every one of them, and the least type that does.

- `infer_admits` `Inference` Every type ever seen at a member fits the type that member was given.
 - `obs_admits` `Inference` Said of one member's observations on their own.
 - `foldJoin_le` `Inference` Folding the type-join over a list of observed types lands above every one of them.
 - `foldJoin_none` `Inference` Once the join is undefined it stays undefined, so a fold that succeeded never passed through failure.
 - `join_idem` `Lattice` A type joined with itself is itself.
 - `le_join_left` `Lattice` The common type is above the first argument.
 - `le_join_right` `Lattice` And above the second.
 - `join_comm` `Lattice` The common type of two types does not depend on which is named first.
 - `le_join_left` `Lattice` *shown above*
 - `le_trans` `Lattice` The type order is transitive.
 - `join_assoc` `Lattice` Nor on how three of them are grouped, which is what makes folding observations order-independent.
- `infer_least` `Inference` And that type is the least one that fits, so nothing is widened further than the data forces.
 - `obs_least` `Inference` Said of one member's observations on their own.
 - `bot_le` `Lattice` The empty type sits below everything.
 - `foldJoin_least` `Inference` And lands below anything else that is above them all.
 - `foldJoin_none` `Inference` *shown above*
 - `join_least` `Lattice` The common type is the *least* type above both -- the tightest answer, not merely an answer.

5. Nullability

`nullability_sound` `Correctness` A column is optional exactly when some document lacked a value there -- neither a missing 'option' nor a gratuitous one.

— `infer_counts` `Counts` Every visit to a table is accounted for at each of its members: a value, an explicit null, or an absence, and nothing else.

— `countsFit_nil` `Counts` Observing nothing trivially adds up.

— `except_bind_ok` `Sorted` *shown above*

— `fold_countsFit` `Counts` So folding a whole corpus keeps them adding up.

— `merge_countsFit` `Counts` Merging two documents' counts keeps them adding up, since visits add on both sides too.

— `mem_mergeBy` `Counts` A key is in a merge exactly when it is in one of the two sides.

— `tableObs_merge_fits` `Counts` The same, said of one table.

— `mem_mergeBy` `Counts` *shown above*

— `inferFinish_counts` `Counts` The final pass, which fills in the absence counts by subtraction, produces counts that add up.

— `except_bind_ok` `Sorted` *shown above*

— `mem_of_mapM_ok` `Sorted` *shown above*

— `observeDocument_fits` `Counts` One document leaves room at every member: no member is counted more times than its table was visited.

— `TablesOk_nil` `Merge` *shown above*

— `countsFit_nil` `Counts` *shown above*

— `walk_counts` `Counts` The walk keeps that true, alongside the order invariant.

— `Tables.upsert_ok` `Merge` *shown above*

— `checkDistinct_sound` `Counts` A document whose members repeat is refused, so a table never sees one member twice in a visit.

— `checkDistinct_go_sound` `Counts` The loop that performs that check means what it says.

— `except_bind_ok` `Sorted` *shown above*

— `ne_of_longer` `Counts` A longer path is a different path.

— `not_prefix_of_longer` `Counts` And a longer path is not a prefix of a shorter one.

— `prefix_entry` `Counts` And of itself extended by a map marker.

— `prefix_member` `Counts` A path is a prefix of itself extended by a member.

— `recordMember_fits` `Counts` Recording one observation keeps the counts fitting, given the member had room.

— `lookup_of_mem_nodup` `Counts` And with distinct keys, what it looks up is the entry that is there.

— `mem_insertBy` `Counts` What inserting one entry leaves in the list.

— `mem_upsert` `Counts` What recording a table leaves in the list.

— `mem_insertBy` `Counts` *shown above*

— `tables_nodup` `Counts` Observations kept in path order repeat no path.

— `pathOrder` `Order` *shown above*

— `sortedBy_nodup` `Sorted` A list sorted by a strict order repeats no key.

— `below_not_mem` `Sorted` A key below everything in a list does not appear in it.

— `recordMember_ok` `Merge` *shown above*

— `recordMember_room` `Counts` Recording at one member leaves room at the others.

— `mem_insertBy` `Counts` *shown above*

	<code>upsert_lookup_self</code> Counts Recording a table and looking it up gives the updated table. <code>lookup_map_self</code> Counts Rewriting one entry in place changes what that key looks up.
	<code>room_frame</code> Counts Neither does anything that leaves this table's subtree alone.
	<code>room_mono</code> Counts Room for a set of members is room for any part of it.
	<code>room_upsert_ne</code> Counts Touching a different table does not consume a member's room.
	<code>upsert_lookup_ne</code> Counts And leaves every other table's lookup alone. <code>insertBy_lookup_ne</code> Counts And leaves every other key's lookup alone. <code>lookup_map_upd</code> Counts And leaves the others alone.
	<code>upsert_fits_keep</code> Counts An update that leaves the members alone and never lowers the visit count keeps them fitting.
	<code>mem_upsert</code> Counts <i>shown above</i>
	<code>upsert_visit_room</code> Counts And one that raises the visit count by one makes room for a fresh observation at every member.
	<code>lookup_mem</code> Counts A key that looks something up is in the list. <code>upsert_lookup_self</code> Counts <i>shown above</i> <code>upsert_lookup_self_none</code> Counts Even when it was not there before. <code>insertBy_lookup_self</code> Counts Inserting at a key and looking it up gives the combined value.
	<code>walk_frame</code> Counts The walk only ever touches the subtree it was pointed at.
	<code>except_bind_ok</code> Sorted <i>shown above</i> <code>frame_refl</code> Counts Changing nothing leaves everything outside a subtree alone. <code>frame_trans</code> Counts And two such changes compose. <code>frame_upsert</code> Counts Updating a table inside a subtree leaves everything outside it alone. <code>upsert_lookup_ne</code> Counts <i>shown above</i> <code>frame_widen</code> Counts What is left alone outside a subtree is left alone outside any smaller one. <code>prefix_elem</code> Counts And of itself extended by an array marker. <code>prefix_entry</code> Counts <i>shown above</i> <code>prefix_member</code> Counts <i>shown above</i> <code>recordMember_frame</code> Counts Recording one observation only touches its own table. <code>frame_upsert</code> Counts <i>shown above</i>
	<code>walk_frame_elems</code> Counts The same, said of an array's elements.
	<code>except_bind_ok</code> Sorted <i>shown above</i> <code>frame_refl</code> Counts <i>shown above</i> <code>frame_trans</code> Counts <i>shown above</i> <code>frame_upsert</code> Counts <i>shown above</i> <code>frame_widen</code> Counts <i>shown above</i> <code>prefix_elem</code> Counts <i>shown above</i> <code>prefix_entry</code> Counts <i>shown above</i> <code>prefix_member</code> Counts <i>shown above</i> <code>recordMember_frame</code> Counts <i>shown above</i>
	<code>walk_frame_entries</code> Counts And of a map's entries.
	<code>except_bind_ok</code> Sorted <i>shown above</i> <code>frame_refl</code> Counts <i>shown above</i> <code>frame_trans</code> Counts <i>shown above</i> <code>frame_upsert</code> Counts <i>shown above</i>

frame_widen	Counts	shown above
prefix_elem	Counts	shown above
prefix_entry	Counts	shown above
prefix_member	Counts	shown above
recordMember_frame	Counts	shown above

04

Every property, in full

All 163 results, grouped by the area they belong to: what each says in plain words, and the Lean statement verbatim.

Correctness ⁶

The five named parts, and the theorem that conjoins them.

nullability_sound

A column is optional exactly when some document lacked a value there -- neither a missing `option` nor a gratuitous one.

```
theorem nullability_sound (cfg : Config) (ds : List Doc) (ts : Tables)
  (hinf : inferCorpus cfg ds = .ok ts) :
  ∀ p t, (p, t) ∈ ts → ∀ k o, (k, o) ∈ t.members →
    o.values + o.nulls + o.absent = t.visits
  ∧ (o.nullable = true ↔ o.values < t.visits)
```

pipeline_correct

Everything below in one statement: for a corpus inference accepted and a schema the generator accepted, the signature is well formed, the schema is canonical, the structure is preserved and the types are principal.

```
theorem pipeline_correct (cfg : Config) (nm : Naming) (ds : List Doc)
  (ts : Tables) (f : File)
  (hinf : inferCorpus cfg ds = .ok ts)
  (hgen : gen nm (toSchema ts) = .ok f) :
  f.WellFormed
  ∧ (∀ es ts', ds.Perm es → inferCorpus cfg es = .ok ts' → ts = ts')
  ∧ ((∀ p q, CollEdge (toSchema ts) p q →
      FileCollEdge f (moduleName nm p) (moduleName nm q))
    ∧ (∀ p q, RefEdge (toSchema ts) p q →
      FileRefEdge f (moduleName nm p) (moduleName nm q))
    ∧ (∀ t u, t ∈ toSchema ts → u ∈ toSchema ts →
      FileCollEdge f (moduleName nm t.path) (moduleName nm u.path) →
      CollEdge (toSchema ts) t.path u.path)
    ∧ (∀ t u, t ∈ toSchema ts → u ∈ toSchema ts →
      FileRefEdge f (moduleName nm t.path) (moduleName nm u.path) →
      RefEdge (toSchema ts) t.path u.path)
    ∧ (∀ t u, t ∈ toSchema ts → u ∈ toSchema ts →
      moduleName nm t.path = moduleName nm u.path → t.path = u.path)
    ∧ Rooted (toSchema ts))
  ∧ (∀ p t, (p, t) ∈ ts → ∀ k o, (k, o) ∈ t.members → ∀ τ, o.joined = some τ →
    (∀ ty, ty ∈ o.seen → ty ⊆ τ) ∧ (∀ σ, (∀ ty, ty ∈ o.seen → ty ⊆ σ) → τ ⊆ σ))
  ∧ (∀ p t, (p, t) ∈ ts → ∀ k o, (k, o) ∈ t.members →
    o.values + o.nulls + o.absent = t.visits
    ∧ (o.nullable = true ↔ o.values < t.visits))
```

schema_canonical

The schema is determined by the corpus as a collection of documents, not by the order they arrived in -- shuffle it and the answer is equal, not merely equivalent.

```
theorem schema_canonical (cfg : Config) (ds : List Doc) (ts : Tables)
  (hinf : inferCorpus cfg ds = .ok ts) :
  ∀ es ts', ds.Perm es → inferCorpus cfg es = .ok ts' → ts = ts'
```

signature_wellFormed

The emitted signature repeats no module name, no value name within a module and no field name within a record.

```
theorem signature_wellFormed (nm : Naming) (s : Schema) (f : File)
  (hgen : gen nm s = .ok f) : f.WellFormed
```

structure_preserved

The tree the documents induce and the graph a reader finds in the generated signatures are the same graph, and it is rooted.

```
theorem structure_preserved (nm : Naming) (s : Schema) (f : File)
  (hgen : gen nm s = .ok f) :
  -- nothing lost
  (∀ p q, CollEdge s p q → FileCollEdge f (moduleName nm p) (moduleName nm q))
  ∧ (∀ p q, RefEdge s p q → FileRefEdge f (moduleName nm p) (moduleName nm q))
  -- nothing invented
  ∧ (∀ t u, t ∈ s → u ∈ s →
    FileCollEdge f (moduleName nm t.path) (moduleName nm u.path) →
    CollEdge s t.path u.path)
  ∧ (∀ t u, t ∈ s → u ∈ s →
    FileRefEdge f (moduleName nm t.path) (moduleName nm u.path) →
    RefEdge s t.path u.path)
  -- the same graph, not a quotient of it
  ∧ (∀ t u, t ∈ s → u ∈ s →
    moduleName nm t.path = moduleName nm u.path → t.path = u.path)
  -- and rooted
  ∧ Rooted s
```

types_principal

Each member is given the principal type of the values seen there: one that admits every one of them, and the least type that does.

```
theorem types_principal (cfg : Config) (ds : List Doc) (ts : Tables)
  (hinf : inferCorpus cfg ds = .ok ts) :
  ∀ p t, (p, t) ∈ ts → ∀ k o, (k, o) ∈ t.members → ∀ τ, o.joined = some τ →
  (∀ ty, ty ∈ o.seen → ty ⊆ τ) ∧ (∀ σ, (∀ ty, ty ∈ o.seen → ty ⊆ σ) → τ ⊆ σ)
```

Tree 42

That the nesting of the JSON and the shape of the generated signatures are the same tree. The largest group, because the correspondence has to be proved in both directions and the generator taken apart to do it.

baseCols_ref

Of the columns a table's position adds, only the back reference is a key into another table.

```
theorem baseCols_ref {nm : Naming} {t : Table} {kt : Ty} {c : Col} {p : Path}
  (hkt : keyTyOf t = .ok kt) (hc : c ∈ baseCols nm t kt) (hty : c.ty = .ref p) :
  t.parent = some p
```

colTyExpr_ref

Only a reference column is typed as another module's key; a collection becomes a list and a scalar a scalar.

```
theorem colTyExpr_ref {nm : Naming} {c : Col} {a : String}
  (h : colTyExpr nm c = TyExpr.qualified a "id" v
    colTyExpr nm c = (TyExpr.qualified a "id").option) :
  ∃ p, c.ty = .ref p ∧ a = moduleName nm p
```

find?_id_eq

With distinct member names, looking up the key finds the column that is there to be found.

```
theorem find?_id_eq : ∀ {l : List Column}, (l.map Column.name).Nodup →
  ∀ {c : Column}, c ∈ l → c.name = "id" →
  l.find? (fun x => x.name == "id") = some c
```

genLoader_decls

A loader has no signature at all, so it carries no edges.

```
theorem genLoader_decls {s : Schema} {nm : Naming} {t : Table} {m : Module}
  (h : genLoader s nm t = .ok m) : m.decls = []
```

genModule_accessor

Every column of the layout becomes an accessor in the signature.

```
theorem genModule_accessor {nm : Naming} {t : Table} {m : Module} {cols : List Col} {c : Col}
  (h : genModule nm t = .ok m) (hl : layoutOf nm t = .ok cols) (hc : c ∈ cols) :
  Decl.value c.name (.arrow (.named "t") (colTyExpr nm c)) ∈ m.decls
```

genModule_accessor_only

And accessors are the only declarations that take a row and return another module's key.

```
theorem genModule_accessor_only {nm : Naming} {t : Table} {m : Module}
  {n : String} {X : TyExpr}
  (h : genModule nm t = .ok m)
  (hd : Decl.value n ((TyExpr.named "t").arrow X) ∈ m.decls)
  (hX : ∀ Y Z, X ≠ TyExpr.arrow Y Z) :
  ∃ cols c, layoutOf nm t = .ok cols ∧ c ∈ cols ∧ X = colTyExpr nm c
```

genModule_layout

A module was only generated if its column layout was computed first.

```
theorem genModule_layout {nm : Naming} {t : Table} {m : Module}
  (h : genModule nm t = .ok m) : ∃ cols, layoutOf nm t = .ok cols
```

genModule_lookup

A table whose rows sit in a collection declares the lookup that finds them from the holder's key.

```
theorem genModule_lookup {nm : Naming} {t : Table} {m : Module} {pp : Path}
  (h : genModule nm t = .ok m) (hp : t.parent = some pp) :
  Decl.value (childLookupName nm pp)
    (.arrow (.qualified (moduleName nm pp) "id") (.list (.named "t")))) ∈ m.decls
```

genModule_lookup_only

And that is the only declaration of that shape, so a reader who finds one has found a parent and not something that merely resembles one.

```
theorem genModule_lookup_only {nm : Naming} {t : Table} {m : Module} {a n : String}
  (h : genModule nm t = .ok m)
  (hd : Decl.value n ((TyExpr.qualified a "id").arrow (TyExpr.list (TyExpr.named "t"))))
    ∈ m.decls) :
  ∃ pp, t.parent = some pp ∧ a = moduleName nm pp
```

genModule_name

A table's module is named after that table.

```
theorem genModule_name {nm : Naming} {t : Table} {m : Module}
  (h : genModule nm t = .ok m) : m.name = moduleName nm t.path
```

genRegistry_decls

Neither does the registry.

```
theorem genRegistry_decls {s : Schema} {nm : Naming} {m : Module}
  (h : genRegistry s nm = .ok m) : m.decls = []
```

gen_faithful

The generator checks its input has the shape a corpus produces, so the tree properties hold of whatever it is handed.

```
theorem gen_faithful (nm : Naming) (s : Schema) (f : File) (h : gen nm s = .ok f) :
  Faithful s
```

gen_inv

Generation is the raw pass followed by a check that hands back its input unchanged.

```
theorem gen_inv {nm : Naming} {s : Schema} {f : File} (h : gen nm s = .ok f) :
  genRaw nm s = .ok f
```

gen_moduleName_injOn

Two different tables never get the same module name, so the file's graph is the same graph and not a collapse of it.

```
theorem gen_moduleName_injOn (nm : Naming) (s : Schema) (f : File)
  (h : gen nm s = .ok f) :
  ∀ t u, t ∈ s → u ∈ s → moduleName nm t.path = moduleName nm u.path → t.path = u.path
```

gen_module_of

Every table of the schema has a module in the output.

```
theorem gen_module_of {nm : Naming} {s : Schema} {f : File} {t : Table}
  (h : gen nm s = .ok f) (ht : t ∈ s) :
  ∃ m, m ∈ f.modules ∧ genModule nm t = .ok m
```

gen_module_source

Every module carrying any declaration at all is some table's.

```
theorem gen_module_source {nm : Naming} {s : Schema} {f : File} {m : Module}
  (h : gen nm s = .ok f) (hm : m ∈ f.modules) (hd : m.decls ≠ []) :
  ∃ t, t ∈ s ∧ genModule nm t = .ok m
```

gen_modules

Every module in the output came from one of the three generation passes.

```
theorem gen_modules {nm : Naming} {s : Schema} {f : File} (h : gen nm s = .ok f) :
  ∃ units loaders registry,
    (orderTables s).mapM (genModule nm) = .ok units ∧
    (orderTables s).mapM (genLoader s nm) = .ok loaders ∧
    genRegistry s nm = .ok registry ∧
    f.modules = units ++ loaders ++ [registry]
```

gen_rooted

Every table is reachable from the root, so walking the signatures from the root module enumerates the whole schema.

```
theorem gen_rooted (nm : Naming) (s : Schema) (f : File) (h : gen nm s = .ok f) :
  Rooted s
```

gen_schemaOk

If generation succeeded, the schema passed that check.

```
theorem gen_schemaOk {nm : Naming} {s : Schema} {f : File} (h : gen nm s = .ok f) :
  schemaOkB s = true
```

gen_tree_complete

Nothing in the document's tree is lost: every nesting the documents induce is declared in the generated signatures.

```
theorem gen_tree_complete (nm : Naming) (s : Schema) (f : File)
  (h : gen nm s = .ok f) :
  (∀ p q, CollEdge s p q → FileCollEdge f (moduleName nm p) (moduleName nm q)) ∧
  (∀ p q, RefEdge s p q → FileRefEdge f (moduleName nm p) (moduleName nm q))
```

gen_tree_complete_coll

The half of that for collections: a table whose rows sit inside another declares the lookup that finds them.

```
theorem gen_tree_complete_coll (nm : Naming) (s : Schema) (f : File)
  (h : gen nm s = .ok f) :
  ∀ p q, CollEdge s p q → FileCollEdge f (moduleName nm p) (moduleName nm q)
```

gen_tree_complete_ref

The half for references: a member that was an object and became a table leaves an accessor holding that table's key.

```
theorem gen_tree_complete_ref (nm : Naming) (s : Schema) (f : File)
  (h : gen nm s = .ok f) :
  ∀ p q, RefEdge s p q → FileRefEdge f (moduleName nm p) (moduleName nm q)
```

gen_tree_sound

Nothing is invented: every edge a reader finds between two tables is one the documents actually induced.

```
theorem gen_tree_sound (nm : Naming) (s : Schema) (f : File)
  (h : gen nm s = .ok f) :
  (∀ t u, t ∈ s → u ∈ s →
    FileCollEdge f (moduleName nm t.path) (moduleName nm u.path) →
    CollEdge s t.path u.path) ∧
  (∀ t u, t ∈ s → u ∈ s →
    FileRefEdge f (moduleName nm t.path) (moduleName nm u.path) →
    RefEdge s t.path u.path)
```

inj_of_nodup_map

If a list's keys are all distinct then two elements with the same key are the same element.

```
theorem inj_of_nodup_map {α β : Type} {f : α → β} :
  ∀ {l : List α}, (l.map f).Nodup → ∀ {a b : α}, a ∈ l → b ∈ l → f a = f b → a = b
```

keyTyOf_id_not_ref

A member called `id` that is an object is refused outright, so a reference never hides in the key.

```
theorem keyTyOf_id_not_ref {t : Table} {kt : Ty} {c : Column} {q : Path}
  (h : keyTyOf t = .ok kt) (hn : (t.columns.map Column.name).Nodup)
  (hc : c ∈ t.columns) (hid : c.name = "id") (hty : c.field.ty = .ref q) : False
```

keyTyOf_not_ref

A table's key is never an object.

```
theorem keyTyOf_not_ref {t : Table} {kt : Ty} {p : Path}
  (h : keyTyOf t = .ok kt) : kt ≠ .ref p
```

layoutOf_keyTy

If the layout was computed, the key's type was settled first.

```
theorem layoutOf_keyTy {nm : Naming} {t : Table} {cols : List Col}
  (h : layoutOf nm t = .ok cols) : ∃ kt, keyTyOf t = .ok kt
```

layoutOf_mapM

If the layout was computed, every member was considered.

```
theorem layoutOf_mapM {nm : Naming} {t : Table} {cols : List Col}
  (h : layoutOf nm t = .ok cols) :
  ∃ ms, t.columns.mapM (memberCol (generatedNames nm t)) = .ok ms
```

layoutOf_member

A column produced for a member of the table reaches the layout.

```
theorem layoutOf_member {nm : Naming} {t : Table} {cols : List Col}
  {c : Column} {col : Col}
  (h : layoutOf nm t = .ok cols) (hc : c ∈ t.columns)
  (hmc : memberCol (generatedNames nm t) c = .ok (some col)) : col ∈ cols
```

mem_layoutOf

Every member that is neither the table's key nor a collection becomes a column, under its mangled name, at the type inference gave it.

```
theorem mem_layoutOf {nm : Naming} {t : Table} {cols : List Col} {c : Col}
  (h : layoutOf nm t = .ok cols) (hc : c ∈ cols) :
  (∃ kt, keyTyOf t = .ok kt ∧ c ∈ baseCols nm t kt) ∨
  (∃ col, col ∈ t.columns ∧ memberCol (generatedNames nm t) col = .ok (some c))
```

mem_orderGo_complete

And it loses none, given that no two tables share a path.

```
theorem mem_orderGo_complete {t : Table} :
  ∀ (fuel : Nat) (pending acc : List Table),
    (pending.map Table.path).Nodup →
    (t ∈ pending ∨ t ∈ acc) → t ∈ orderGo fuel pending acc
```

mem_orderGo_sound

The compile-order sort invents no tables.

```
theorem mem_orderGo_sound {t : Table} :
  ∀ (fuel : Nat) (pending acc : List Table),
    t ∈ orderGo fuel pending acc → t ∈ pending ∨ t ∈ acc
```

mem_orderTables_complete

The same, said of the sort as a whole.

```
theorem mem_orderTables_complete {t : Table} {s : Schema}
  (hn : (s.map Table.path).Nodup) (h : t ∈ s) : t ∈ orderTables s
```

mem_orderTables_sound

The same, said of the sort as a whole.

```
theorem mem_orderTables_sound {t : Table} {s : Schema} (h : t ∈ orderTables s) : t ∈ s
```

memberCol_ref

A member that is an object and is not the key becomes a column holding that object's key.

```
theorem memberCol_ref {gen : List String} {c : Column} {q : Path} {o : Option Col}
  (hok : memberCol gen c = .ok o) (hid : ¬ (c.name == idColumn) = true)
  (hty : c.field.ty = .ref q) :
  o = some { name := mangle c.name, ty := .ref q
    , nullable := c.field.nullable, kind := .member c.name }
```

memberCol_ty

A member column keeps the member's own type.

```
theorem memberCol_ty {gen : List String} {c : Column} {col : Col}
  (h : memberCol gen c = .ok (some col)) : col.ty = c.field.ty
```

noClash_modules

The duplicate-name check hands back its input unchanged.

```
theorem noClash_modules {mods : List Module} {f : File}
  (h : noClash mods = .ok f) : f.modules = mods
```

nodupPaths_sound

The check that no two tables share a path means what it says.

```
theorem nodupPaths_sound : ∀ {l : List Path}, nodupPaths l = true → l.Nodup
```

path_mem

A path the check found in the schema is the path of a table of it.

```
private theorem path_mem {s : Schema} {p : Path}
  (h : (s.map Table.path).contains p = true) : ∃ u, u ∈ s ∧ u.path = p
```

reachGo_sound

So does the whole search.

```
theorem reachGo_sound {s : Schema} : ∀ (fuel : Nat) (acc : List Path),
  (∀ p, p ∈ acc → Reaches s p) → ∀ p, p ∈ reachGo s fuel acc → Reaches s p
```

reachable_sound

Everything the search reports is genuinely reachable from the root.

```
theorem reachable_sound {s : Schema} : ∀ p, p ∈ reachable s → Reaches s p
```

stepReach_sound

One round of the reachability search only ever reports tables genuinely one edge further out.

```
theorem stepReach_sound {s : Schema} {acc : List Path}
  (hacc : ∀ p, p ∈ acc → Reaches s p) : ∀ p, p ∈ stepReach s acc → Reaches s p
```

Inference ¹¹

That the corpus determines the schema, and that each member's type is the tightest one its values allow.

foldJoin_le

Folding the type-join over a list of observed types lands above every one of them.

```
theorem foldJoin_le : ∀ {l : List Ty} {a τ : Ty},
  l.foldl1 (fun acc t => acc.bind (Ty.join · t)) (some a) = some τ →
  a ⊆ τ ∧ ∀ t, t ∈ l → t ⊆ τ
```

foldJoin_least

And lands below anything else that is above them all.

```
theorem foldJoin_least : ∀ {l : List Ty} {a τ σ : Ty},
  l.foldl1 (fun acc t => acc.bind (Ty.join · t)) (some a) = some τ →
  a ⊆ σ → (∀ t, t ∈ l → t ⊆ σ) → τ ⊆ σ
```

foldJoin_none

Once the join is undefined it stays undefined, so a fold that succeeded never passed through failure.

```
theorem foldJoin_none (l : List Ty) :
  l.foldl1 (fun acc t => acc.bind (Ty.join · t)) none = none
```

foldl_perm_on

The same, where the laws hold only of values satisfying an invariant.

```
theorem foldl_perm_on {α β : Type} {f : β → α → β} {P : β → Prop} {Q : α → Prop}
  (hP : ∀ b x, P b → Q x → P (f b x))
  (hrc : ∀ b x y, P b → Q x → Q y → f (f b x) y = f (f b y) x) :
  ∀ {l1 l2 : List α}, l1.Perm l2 → (∀ x, x ∈ l1 → Q x) →
  ∀ b, P b → l1.foldl1 f b = l2.foldl1 f b
```

infer_admits

Every type ever seen at a member fits the type that member was given.

```
theorem infer_admits (cfg : Config) (ds : List Doc) (ts : Tables)
  (h : inferCorpus cfg ds = .ok ts) :
  ∀ p t, (p, t) ∈ ts → ∀ k o, (k, o) ∈ t.members →
  ∀ τ, o.joined = some τ → ∀ ty, ty ∈ o.seen → ty ⊆ τ
```

infer_least

And that type is the least one that fits, so nothing is widened further than the data forces.

```
theorem infer_least (cfg : Config) (ds : List Doc) (ts : Tables)
  (h : inferCorpus cfg ds = .ok ts) :
  ∀ p t, (p, t) ∈ ts → ∀ k o, (k, o) ∈ t.members →
  ∀ τ σ, o.joined = some τ → (∀ ty, ty ∈ o.seen → ty ⊆ σ) → τ ⊆ σ
```

infer_perm

The schema is a property of the corpus and not of the order its documents arrived in: shuffle the corpus and you get the same answer.

```
theorem infer_perm (cfg : Config) (ds es : List Doc) (h : ds.Perm es)
  {ta tb : Tables} (hda : inferCorpus cfg ds = .ok ta)
  (hdb : inferCorpus cfg es = .ok tb) :
  ta = tb
```

mapM_ok_of_all

If every element succeeds then the whole list does.

```
theorem mapM_ok_of_all {α β ε : Type} {f : α → Except ε β} :
  ∀ {l : List α}, (∀ x, x ∈ l → ∃ b, f x = .ok b) → ∃ r, l.mapM f = .ok r
```

mapM_perm

Permuted inputs give permuted outputs, when both succeed.

```
theorem mapM_perm {α β ε : Type} {f : α → Except ε β} :  
  ∀ {l₁ l₂ : List α}, l₁.Perm l₂ → ∀ {r₁ r₂ : List β},  
    l₁.mapM f = .ok r₁ → l₂.mapM f = .ok r₂ → r₁.Perm r₂
```

obs_admits

Said of one member's observations on their own.

```
theorem obs_admits {o : Obs} {τ : Ty} (h : o.joined = some τ) :  
  ∀ t, t ∈ o.seen → t ⊆ τ
```

obs_least

Said of one member's observations on their own.

```
theorem obs_least {o : Obs} {τ σ : Ty} (h : o.joined = some τ)  
  (hσ : ∀ t, t ∈ o.seen → t ⊆ σ) : τ ⊆ σ
```

Counts ⁴³

That every visit to a table is accounted for at each of its members, which is what makes a column's `option` mean something. The largest single group, because the invariant has to survive the walk, the merge and the final pass that fills the absence counts in.

checkDistinct_go_sound

The loop that performs that check means what it says.

```
theorem checkDistinct_go_sound (p : Path) :  
  ∀ (ms : List (String × Doc)) (seen : List String),  
    checkDistinct.go p seen ms = .ok () →  
      (ms.map Prod.fst).Nodup ∧ ∀ k ∈ ms.map Prod.fst, k ∉ seen  
| [], _, _ => (by simp, by simp)  
| (k, _) :: tl, seen, h => by  
  simp only [checkDistinct.go] at h  
  split at h  
  · exact absurd h (by simp)  
  · next hc =>  
      obtain (hnd, hns) := checkDistinct_go_sound p tl (k :: seen) h  
      have hk : k ∉ tl.map Prod.fst
```

checkDistinct_sound

A document whose members repeat is refused, so a table never sees one member twice in a visit.

```
theorem checkDistinct_sound {p : Path} {ms : List (String × Doc)}  
  (h : checkDistinct p ms = .ok ()) : (ms.map Prod.fst).Nodup
```

countsFit_nil

Observing nothing trivially adds up.

```
theorem countsFit_nil : CountsFit []
```

fold_countsFit

So folding a whole corpus keeps them adding up.

```
theorem fold_countsFit : ∀ l : List Tables, (∀ x, x ∈ l → CountsFit x) →  
  ∀ b, CountsFit b → CountsFit (l.foldl Tables.merge b)
```

frame_refl

Changing nothing leaves everything outside a subtree alone.

```
theorem frame_refl (q : Path) (ts : Tables) : Frame q ts ts
```

frame_trans

And two such changes compose.

```
theorem frame_trans {q : Path} {a b c : Tables}  
  (h1 : Frame q a b) (h2 : Frame q b c) : Frame q a c
```

frame_upsert

Updating a table inside a subtree leaves everything outside it alone.

```
theorem frame_upsert {q p : Path} (hq : q <+: p) (ts : Tables)  
  (f : TableObs → TableObs) : Frame q ts (ts.upsert p f)
```

frame_widen

What is left alone outside a subtree is left alone outside any smaller one.

```
theorem frame_widen {q q' : Path} {a b : Tables}  
  (hpre : q <+: q') (h : Frame q' a b) : Frame q a b
```

inferFinish_counts

The final pass, which fills in the absence counts by subtraction, produces counts that add up.

```
theorem inferFinish_counts {cfg : Config} {ts ts' : Tables}  
  (h : inferFinish cfg ts = .ok ts') (hfit : CountsFit ts) :  
  ∀ p t, (p, t) ∈ ts' → ∀ k o, (k, o) ∈ t.members →  
    o.values + o.nulls + o.absent = t.visits
```

infer_counts

Every visit to a table is accounted for at each of its members: a value, an explicit null, or an absence, and nothing else.

```
theorem infer_counts (cfg : Config) (ds : List Doc) (ts : Tables)  
  (h : inferCorpus cfg ds = .ok ts) :  
  ∀ p t, (p, t) ∈ ts → ∀ k o, (k, o) ∈ t.members →  
    o.values + o.nulls + o.absent = t.visits
```

insertBy_lookup_ne

And leaves every other key's lookup alone.

```
theorem insertBy_lookup_ne {lt eqk : K → K → Bool} {combine : V → V → V}  
  [BEq K] [LawfulBEq K] (heqk : ∀ a b : K, eqk a b = true → a = b)  
  {p q : K} {v : V} (hne : q ≠ p) :  
  ∀ l : List (K × V), (insertBy lt eqk combine p v l).lookup q = l.lookup q
```

insertBy_lookup_self

Inserting at a key and looking it up gives the combined value.

```
theorem insertBy_lookup_self {lt eqk : K → K → Bool} [BEq K] [LawfulBEq K]
  (heqk : ∀ a b : K, eqk a b = true → a = b) {p : K} {v : V} :
  ∀ l : List (K × V), l.lookup p = none →
    (insertBy lt eqk (fun _ new => new) p v l).lookup p = some v
```

lookup_map_self

Rewriting one entry in place changes what that key looks up.

```
theorem lookup_map_self {p : Path} {f : TableObs → TableObs} {t : TableObs} :
  ∀ ts : Tables, ts.lookup p = some t →
    (ts.map (fun x => if x.1 == p then (x.1, f x.2) else x)).lookup p = some (f t)
```

lookup_map_upd

And leaves the others alone.

```
theorem lookup_map_upd {p q : Path} {f : TableObs → TableObs} (hne : q ≠ p) :
  ∀ ts : Tables,
    (ts.map (fun x => if x.1 == p then (x.1, f x.2) else x)).lookup q = ts.lookup q
```

lookup_mem

A key that looks something up is in the list.

```
theorem lookup_mem {K V : Type} [BEq K] [LawfulBEq K] {k : K} {v : V} :
  ∀ {l : List (K × V)}, l.lookup k = some v → (k, v) ∈ l
```

lookup_of_mem_nodup

And with distinct keys, what it looks up is the entry that is there.

```
theorem lookup_of_mem_nodup {ts : Tables} {p : Path} {t : TableObs}
  (hnd : (ts.map Prod.fst).Nodup) (hmem : (p, t) ∈ ts) : ts.lookup p = some t
```

mem_insertBy

What inserting one entry leaves in the list.

```
theorem mem_insertBy {lt eqk : K → K → Bool} {combine : V → V → V}
  (heqk : ∀ a b : K, eqk a b = true → a = b) {k : K} {v : V} :
  ∀ {l : List (K × V)} {pr : K × V}, pr ∈ insertBy lt eqk combine k v l →
    pr ∈ l ∨ pr = (k, v) ∨ ∃ o, (k, o) ∈ l ∧ pr = (k, combine o v)
```

mem_mergeBy

A key is in a merge exactly when it is in one of the two sides.

```
theorem mem_mergeBy {K V : Type} (lt eqk : K → K → Bool) {combine : V → V → V}
  (heqk : ∀ a b : K, eqk a b = true → a = b) :
  ∀ a b : List (K × V), ∀ pr : K × V, pr ∈ mergeBy lt eqk combine a b →
    pr ∈ a ∨ pr ∈ b ∨
      ∃ ka va vb, (ka, va) ∈ a ∧ (ka, vb) ∈ b ∧ pr = (ka, combine va vb)
```

mem_upsert

What recording a table leaves in the list.

```
theorem mem_upsert {ts : Tables} {p q : Path} {f : TableObs → TableObs} {t' : TableObs}
  (h : (q, t') ∈ ts.upsert p f) :
  ((q, t') ∈ ts) ∨
  (q = p ∧ ∃ t0, t' = f t0 ∧
   ((p, t0) ∈ ts ∨ (t0 = {} ∧ ts.lookup p = none)))
```

merge_countsFit

Merging two documents' counts keeps them adding up, since visits add on both sides too.

```
theorem merge_countsFit {a b : Tables} (ha : CountsFit a) (hb : CountsFit b) :
  CountsFit (Tables.merge a b)
```

ne_of_longer

A longer path is a different path.

```
theorem ne_of_longer {p q : Path} (h : p.length < q.length) : p ≠ q
```

not_prefix_of_longer

And a longer path is not a prefix of a shorter one.

```
theorem not_prefix_of_longer {p q : Path} (h : p.length < q.length) : ¬ (q <+: p)
```

observeDocument_fits

One document leaves room at every member: no member is counted more times than its table was visited.

```
theorem observeDocument_fits {cfg : Config} {d : Doc} {ts : Tables}
  (h : observeDocument cfg d = .ok ts) : CountsFit ts
```

prefix_elem

And of itself extended by an array marker.

```
theorem prefix_elem (p : Path) : p <+: p.elem
```

prefix_entry

And of itself extended by a map marker.

```
theorem prefix_entry (p : Path) : p <+: p.entry
```

prefix_member

A path is a prefix of itself extended by a member.

```
theorem prefix_member (p : Path) (k : String) : p <+: p.member k
```

recordMember_fits

Recording one observation keeps the counts fitting, given the member had room.

```
theorem recordMember_fits {ts : Tables} {p : Path} {k : String} {s : Seen}
  (hok : TablesOk ts) (hfit : CountsFit ts) (hroom : Room ts p [k]) :
  CountsFit (recordMember ts p k s)
```

recordMember_frame

Recording one observation only touches its own table.

```
theorem recordMember_frame {q p : Path} (hq : q <+= p) (ts : Tables)
  (k : String) (s : Seen) : Frame q ts (recordMember ts p k s)
```

recordMember_room

Recording at one member leaves room at the others.

```
theorem recordMember_room {ts : Tables} {p : Path} {k : String} {s : Seen}
  {ks : List String} (hne :  $\forall k' \in ks, k' \neq k$ ) (hroom : Room ts p (k :: ks)) :
  Room (recordMember ts p k s) p ks
```

room_frame

Neither does anything that leaves this table's subtree alone.

```
theorem room_frame {q p : Path} {ts ts' : Tables} {ks : List String}
  (hf : Frame q ts ts') (hnp :  $\neg (q <+= p)$ ) (h : Room ts p ks) : Room ts' p ks :=
  let (t, hlk, hv, hrm)
```

room_mono

Room for a set of members is room for any part of it.

```
theorem room_mono {ts : Tables} {p : Path} {ks ks' : List String}
  (hsub :  $\forall k \in ks', k \in ks$ ) (h : Room ts p ks) : Room ts p ks' :=
  let (t, hlk, hv, hrm)
```

room_upsert_ne

Touching a different table does not consume a member's room.

```
theorem room_upsert_ne {ts : Tables} {p r : Path} {ks : List String}
  {f : TableObs → TableObs} (hne :  $p \neq r$ ) (h : Room ts p ks) :
  Room (ts.upsert r f) p ks :=
  let (t, hlk, hv, hrm)
```

tableObs_merge_fits

The same, said of one table.

```
theorem tableObs_merge_fits {a b : TableObs} (ha : TableFits a) (hb : TableFits b) :
  TableFits (TableObs.merge a b)
```

tables_nodup

Observations kept in path order repeat no path.

```
theorem tables_nodup {ts : Tables} (hok : TablesOk ts) : (ts.map Prod.fst).Nodup
```

upsert_fits_keep

An update that leaves the members alone and never lowers the visit count keeps them fitting.

```
theorem upsert_fits_keep {ts : Tables} {p : Path} {f : TableObs → TableObs}
  (hmem :  $\forall t, (f t).members = t.members$ ) (hvis :  $\forall t, t.visits \leq (f t).visits$ )
  (hfit : CountsFit ts) : CountsFit (ts.upsert p f)
```

upsert_lookup_ne

And leaves every other table's lookup alone.

```
theorem upsert_lookup_ne {ts : Tables} {p q : Path} {f : TableObs → TableObs}
  (hne : q ≠ p) : (ts.upsert p f).lookup q = ts.lookup q
```

upsert_lookup_self

Recording a table and looking it up gives the updated table.

```
theorem upsert_lookup_self {ts : Tables} {p : Path} {f : TableObs → TableObs}
  {t : TableObs} (h1k : ts.lookup p = some t) :
  (ts.upsert p f).lookup p = some (f t)
```

upsert_lookup_self_none

Even when it was not there before.

```
theorem upsert_lookup_self_none {ts : Tables} {p : Path} {f : TableObs → TableObs}
  (h1k : ts.lookup p = none) : (ts.upsert p f).lookup p = some (f {})
```

upsert_visit_room

And one that raises the visit count by one makes room for a fresh observation at every member.

```
theorem upsert_visit_room {ts : Tables} {p : Path} {f : TableObs → TableObs}
  (hmem : ∀ t, (f t).members = t.members) (hvis : ∀ t, (f t).visits = t.visits + 1)
  (hfit : CountsFit ts) (ks : List String) : Room (ts.upsert p f) p ks
```

walk_counts

The walk keeps that true, alongside the order invariant.

```
theorem walk_counts (cfg : Config) :
  ∀ ts target ic j, ∀ ts', observeObject cfg ts target ic j = .ok ts' →
  TablesOk ts → CountsFit ts → TablesOk ts' ∧ CountsFit ts'
```

walk_frame

The walk only ever touches the subtree it was pointed at.

```
theorem walk_frame (cfg : Config) :
  ∀ ts target inCollection j, ∀ ts',
  observeObject cfg ts target inCollection j = .ok ts' → Frame target ts ts'
```

walk_frame_elems

The same, said of an array's elements.

```
theorem walk_frame_elems (cfg : Config) :
  ∀ ts et raw els, ∀ ts',
  observeElems cfg ts et raw els = .ok ts' → Frame et ts ts'
```

walk_frame_entries

And of a map's entries.

```
theorem walk_frame_entries (cfg : Config) :
  ∀ ts et raw es, ∀ ts',
  observeEntries cfg ts et raw es = .ok ts' → Frame et ts ts'
```

Walk ³

That the traversal which reads the documents keeps its observations in order — the invariant everything in the previous group relies on.

keep

Each record update the walk makes leaves the member list alone.

```
private theorem keep {f : TableObs → TableObs}
  (hm : ∀ t, (f t).members = t.members) : ∀ t, TableObsOk t → TableObsOk (f t)
```

observeDocument_ok

So one document, observed from nothing, comes out in order.

```
theorem observeDocument_ok {cfg : Config} {d : Doc} {ts : Tables}
  (h : observeDocument cfg d = .ok ts) : TablesOk ts
```

observe_ok

Everything the inference walk records is kept in key order -- tables by path, members by name, types by their order.

```
theorem observe_ok (cfg : Config) :
  ∀ ts target inCollection j, TablesOk ts → ∀ ts',
    observeObject cfg ts target inCollection j = .ok ts' → TablesOk ts'
```

Merge ¹⁴

That combining two documents' observations is commutative and associative, at each of the three levels the data has.

Obs.merge_assoc

And one level down again.

```
theorem Obs.merge_assoc {a b c : Obs} (ha : ObsOk a) (hb : ObsOk b) (hc : ObsOk c) :
  Obs.merge (Obs.merge a b) c = Obs.merge a (Obs.merge b c)
```

Obs.merge_comm

And one level down again, for a single member's: counts add, type sets union, flags disjoint.

```
theorem Obs.merge_comm {a b : Obs} (ha : ObsOk a) (hb : ObsOk b) :
  Obs.merge a b = Obs.merge b a
```

Obs.merge_ok

And one level down again.

```
theorem Obs.merge_ok {a b : Obs} (ha : ObsOk a) (hb : ObsOk b) :
  ObsOk (Obs.merge a b)
```

TableObs.merge_assoc

The same, one level down.

```
theorem TableObs.merge_assoc {a b c : TableObs}
  (ha : TableObsOk a) (hb : TableObsOk b) (hc : TableObsOk c) :
  TableObs.merge (TableObs.merge a b) c = TableObs.merge a (TableObs.merge b c)
```

TableObs.merge_comm

The same, one level down, for a single table's observations.

```
theorem TableObs.merge_comm {a b : TableObs} (ha : TableObsOk a) (hb : TableObsOk b) :  
  TableObs.merge a b = TableObs.merge b a
```

TableObs.merge_ok

The same, one level down.

```
theorem TableObs.merge_ok {a b : TableObs} (ha : TableObsOk a) (hb : TableObsOk b) :  
  TableObsOk (TableObs.merge a b) :=  
  (mergeBy_sorted strOrder _ _ ha.1 hb.1,  
   mergeBy_allV (combine := Obs.merge) (P
```

TableObsOk.empty

So is a table with no members yet.

```
theorem TableObsOk.empty : TableObsOk ({} : TableObs)
```

Tables.merge_assoc

And grouping three of them either way gives the same answer.

```
theorem Tables.merge_assoc {a b c : Tables}  
  (ha : TablesOk a) (hb : TablesOk b) (hc : TablesOk c) :  
  Tables.merge (Tables.merge a b) c = Tables.merge a (Tables.merge b c) :=  
  mergeBy_assocOn (combine := TableObs.merge) (P
```

Tables.merge_comm

Merging two documents' observations gives the same answer in either order.

```
theorem Tables.merge_comm {a b : Tables} (ha : TablesOk a) (hb : TablesOk b) :  
  Tables.merge a b = Tables.merge b a :=  
  mergeBy_commOn (combine := TableObs.merge) (P
```

Tables.merge_ok

Merging preserves the order invariant.

```
theorem Tables.merge_ok {a b : Tables} (ha : TablesOk a) (hb : TablesOk b) :  
  TablesOk (Tables.merge a b) :=  
  (mergeBy_sorted pathOrder _ _ ha.1 hb.1,  
   mergeBy_allV (combine := TableObs.merge) (P
```

Tables.merge_right_comm

Together: two documents merged into an accumulator commute, which is what the corpus fold needs.

```
theorem Tables.merge_right_comm {b x y : Tables}  
  (hb : TablesOk b) (hx : TablesOk x) (hy : TablesOk y) :  
  Tables.merge (Tables.merge b x) y = Tables.merge (Tables.merge b y) x
```

Tables.upsert_ok

Recording a table keeps everything in order.

```
theorem Tables.upsert_ok {ts : Tables} {p : Path} {f : TableObs → TableObs}  
  (h : TablesOk ts) (hf : ∀ t, TableObsOk t → TableObsOk (f t)) :  
  TablesOk (ts.upsert p f)
```

TablesOk.nil

Observing nothing is trivially in order.

```
theorem TablesOk.nil : TablesOk []
```

recordMember_ok

So does recording one observation at one member.

```
theorem recordMember_ok {ts : Tables} {p : Path} {k : String} {sn : Seen}
  (h : TablesOk ts) : TablesOk (recordMember ts p k sn)
```

Order ₃

That the three things used as keys — member names, table paths, types — are each strictly totally ordered.

pathOrder

So are table paths.

```
theorem pathOrder : StrictOrder Path.lt (· == ·) where
  eq_iff a b
```

strOrder

Member names are strictly totally ordered.

```
theorem strOrder : StrictOrder (fun a b : String => decide (a < b)) (· == ·) where
  eq_iff a b
```

tyOrder

So are types.

```
theorem tyOrder : StrictOrder Ty.lt (· == ·) where
  eq_iff a b
```

Sorted ₂₉

The generic machinery: sorted association lists, and the fact that merging them is characterised entirely by what they look up.

allV_map

And preserves a property of the values.

```
theorem allV_map {P : V → Prop} {g : K × V → K × V} (hg : ∀ p, P p.2 → P (g p).2) :
  ∀ l : List (K × V), AllV P l → AllV P (l.map g)
```

belowK_of_lt

Transitivity carries `below` down from a smaller key.

```
theorem belowK_of_lt {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) {j k : K} :
  ∀ {l : List K}, BelowK lt k l → lt j k = true → BelowK lt j l
```

below_map

So it leaves `below` alone.

```
theorem below_map {lt : K → K → Bool} {j : K} {g : K × V → K × V}
  (hg : ∀ p, (g p).1 = p.1) : ∀ l : List (K × V), Below lt j l → Below lt j (l.map g)
```

below_not_mem

A key below everything in a list does not appear in it.

```
theorem below_not_mem {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) {k : K} :
  ∀ {l : List (K × V)}, Below lt k l → k ∉ l.map Prod.fst
```

below_of_lt

The same, for association lists.

```
theorem below_of_lt {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) {j k : K} :
  ∀ {l : List (K × V)}, Below lt k l → lt j k = true → Below lt j l
```

except_bind_ok

A computation that succeeded had both halves succeed.

```
theorem except_bind_ok {ε α β : Type} {x : Except ε α} {g : α → Except ε β} {b : β}
  (h : (x >=> g) = .ok b) : ∃ a, x = .ok a ∧ g a = .ok b
```

insertBy_allV

And preserves a property of the values.

```
theorem insertBy_allV {lt eqk : K → K → Bool} {combine : V → V → V} {P : V → Prop}
  (hP : ∀ x y, P x → P y → P (combine x y)) {k : K} {v : V} (hv : P v) :
  ∀ l : List (K × V), AllV P l → AllV P (insertBy lt eqk combine k v l)
```

insertBy_below

And keeps a smaller key below it.

```
theorem insertBy_below {lt eqk : K → K → Bool} {combine : V → V → V} {j k : K} {v : V} :
  ∀ l : List (K × V), Below lt j l → lt j k = true →
    Below lt j (insertBy lt eqk combine k v l)
```

insertBy_sorted

Inserting one entry keeps the list ordered.

```
theorem insertBy_sorted {lt eqk : K → K → Bool} {combine : V → V → V}
  (ho : StrictOrder lt eqk) {k : K} {v : V} :
  ∀ l : List (K × V), SortedBy lt l → SortedBy lt (insertBy lt eqk combine k v l)
```

lookupBy_allV

A value looked up satisfies whatever the list's values do.

```
theorem lookupBy_allV {eqk : K → K → Bool} {P : V → Prop} {k : K} {v : V} :
  ∀ {l : List (K × V)}, AllV P l → lookupBy eqk k l = some v → P v
```

lookupBy_of_below

A key below everything in a list is not in it.

```
theorem lookupBy_of_below {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) {k : K} :  
  ∀ {l : List (K × V)}, Below lt k l → lookupBy eqk k l = none
```

mapM_ok_map

A mapped list agrees pointwise with its inputs under any function that agrees on each.

```
theorem mapM_ok_map {α β γ ε : Type} {f : α → Except ε β} {g : β → γ} {k : α → γ}  
  (hgk : ∀ a b, f a = .ok b → g b = k a) :  
  ∀ {l : List α} {r : List β}, l.mapM f = .ok r → r.map g = l.map k
```

mapM_ok_pointwise

And every input reached some output.

```
theorem mapM_ok_pointwise {α β ε : Type} {f : α → Except ε β} :  
  ∀ {l : List α} {r : List β}, l.mapM f = .ok r →  
    ∀ a, a ∈ l → ∃ b, b ∈ r ∧ f a = .ok b
```

memBy_of_below

A key below everything in a set is not in it.

```
theorem memBy_of_below {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) {k : K} :  
  ∀ {l : List K}, BelowK lt k l → memBy eqk k l = false
```

mem_of_mapM_ok

Every output of a mapped list came from some input.

```
theorem mem_of_mapM_ok {α β ε : Type} {f : α → Except ε β} :  
  ∀ {l : List α} {r : List β}, l.mapM f = .ok r → ∀ b, b ∈ r → ∃ a, a ∈ l ∧ f a = .ok b
```

mergeBy_assocOn

Merging is associative, for the same reason.

```
theorem mergeBy_assocOn {lt eqk : K → K → Bool} {combine : V → V → V} {P : V → Prop}  
  (ho : StrictOrder lt eqk)  
  (hP : ∀ x y, P x → P y → P (combine x y))  
  (hca : ∀ x y z, P x → P y → P z → combine (combine x y) z = combine x (combine y z)) :  
  ∀ a b c : List (K × V), SortedBy lt a → SortedBy lt b → SortedBy lt c →  
    AllV P a → AllV P b → AllV P c →  
    mergeBy lt eqk combine (mergeBy lt eqk combine a b) c  
      = mergeBy lt eqk combine a (mergeBy lt eqk combine b c)
```

mergeBy_below

A key below both lists is below their merge.

```
theorem mergeBy_below {lt eqk : K → K → Bool} {combine : V → V → V} {j : K} :  
  ∀ a b : List (K × V), Below lt j a → Below lt j b →  
    Below lt j (mergeBy lt eqk combine a b)
```

mergeBy_commOn

Merging is commutative, because combining lookups is.

```
theorem mergeBy_commOn {lt eqk : K → K → Bool} {combine : V → V → V} {P : V → Prop}
  (ho : StrictOrder lt eqk)
  (hc : ∀ x y, P x → P y → combine x y = combine y x) :
  ∀ a b : List (K × V), SortedBy lt a → SortedBy lt b → AllV P a → AllV P b →
    mergeBy lt eqk combine a b = mergeBy lt eqk combine b a
```

mergeBy_lookup

A merge is exactly its two lookups combined -- this is the whole content of merging, and both laws below read it off.

```
theorem mergeBy_lookup {lt eqk : K → K → Bool} {combine : V → V → V}
  (ho : StrictOrder lt eqk) :
  ∀ a b : List (K × V), SortedBy lt a → SortedBy lt b → ∀ k,
    lookupBy eqk k (mergeBy lt eqk combine a b)
      = combineOpt combine (lookupBy eqk k a) (lookupBy eqk k b)
```

mergeBy_sorted

Merging two ordered lists gives an ordered list.

```
theorem mergeBy_sorted {lt eqk : K → K → Bool} {combine : V → V → V}
  (ho : StrictOrder lt eqk) :
  ∀ a b : List (K × V), SortedBy lt a → SortedBy lt b →
    SortedBy lt (mergeBy lt eqk combine a b)
```

sortedBy_map

Rewriting values in place leaves the order alone.

```
theorem sortedBy_map {lt : K → K → Bool} {g : K × V → K × V}
  (hg : ∀ p, (g p).1 = p.1) :
  ∀ l : List (K × V), SortedBy lt l → SortedBy lt (l.map g)
```

sortedBy_nodup

A list sorted by a strict order repeats no key.

```
theorem sortedBy_nodup {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :
  ∀ l : List (K × V), SortedBy lt l → (l.map Prod.fst).Nodup
```

sortedK_ext

Two sorted sets with the same members are the same list.

```
theorem sortedK_ext {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :
  ∀ {a b : List K}, SortedK lt a → SortedK lt b →
    (∀ k, memBy eqk k a = memBy eqk k b) → a = b
```

sorted_ext

Two sorted lists that agree on every lookup are the same list.

```
theorem sorted_ext {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :
  ∀ {a b : List (K × V)}, SortedBy lt a → SortedBy lt b →
    (∀ k, lookupBy eqk k a = lookupBy eqk k b) → a = b
```

unionBy_assoc

And associative.

```
theorem unionBy_assoc {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :  
  ∀ a b c : List K, SortedK lt a → SortedK lt b → SortedK lt c →  
    unionBy lt eqk (unionBy lt eqk a b) c = unionBy lt eqk a (unionBy lt eqk b c)
```

unionBy_belowK

A key below both sets is below their union.

```
theorem unionBy_belowK {lt eqk : K → K → Bool} {j : K} :  
  ∀ a b : List K, BelowK lt j a → BelowK lt j b → BelowK lt j (unionBy lt eqk a b)
```

unionBy_comm

The same for a set: union is commutative.

```
theorem unionBy_comm {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :  
  ∀ a b, unionBy lt eqk a b = unionBy lt eqk b a
```

unionBy_mem

A union is exactly the two memberships, disjointed.

```
theorem unionBy_mem {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :  
  ∀ a b : List K, SortedK lt a → SortedK lt b → ∀ k,  
    memBy eqk k (unionBy lt eqk a b) = (memBy eqk k a || memBy eqk k b)
```

unionBy_sorted

And preserves order.

```
theorem unionBy_sorted {lt eqk : K → K → Bool} (ho : StrictOrder lt eqk) :  
  ∀ a b : List K, SortedK lt a → SortedK lt b → SortedK lt (unionBy lt eqk a b)
```

Lattice ⁸

That the type order is a lattice and the join is the least upper bound, which is what makes an inferred type the tightest one.

bot_le

The empty type sits below everything.

```
theorem bot_le (a : Ty) : Ty.bot ⊆ a
```

join_assoc

Nor on how three of them are grouped, which is what makes folding observations order-independent.

```
theorem join_assoc (a b c : Ty) :  
  (Ty.join a b).bind (fun ab => Ty.join ab c)  
  = (Ty.join b c).bind (fun bc => Ty.join a bc)
```

join_comm

The common type of two types does not depend on which is named first.

```
theorem join_comm (a b : Ty) : Ty.join a b = Ty.join b a
```

join_idem

A type joined with itself is itself.

```
theorem join_idem (a : Ty) : Ty.join a a = some a
```

join_least

The common type is the *least* type above both -- the tightest answer, not merely an answer.

```
theorem join_least {a b c d : Ty} (h : Ty.join a b = some c) (ha : a  $\sqsubseteq$  d) (hb : b  $\sqsubseteq$  d) :  
  c  $\sqsubseteq$  d
```

le_join_left

The common type is above the first argument.

```
theorem le_join_left {a b c : Ty} (h : Ty.join a b = some c) : a  $\sqsubseteq$  c
```

le_join_right

And above the second.

```
theorem le_join_right {a b c : Ty} (h : Ty.join a b = some c) : b  $\sqsubseteq$  c
```

le_trans

The type order is transitive.

```
theorem le_trans {a b c : Ty} (h1 : a  $\sqsubseteq$  b) (h2 : b  $\sqsubseteq$  c) : a  $\sqsubseteq$  c
```

Wellformed ⁴

That the emitted signature never repeats a name.

certify_wellFormed

The generator checks its own output before returning it.

```
theorem certify_wellFormed {f g : File} (h : certify f = .ok g) : g.WellFormed
```

gen_wellFormed

Every generated signature is well formed: no module, value or record field name repeats.

```
theorem gen_wellFormed (nm : Naming) (s : Schema) (f : File) (h : gen nm s = .ok f) :  
  f.WellFormed
```

nodupNames_sound

And so does its distinctness test.

```
theorem nodupNames_sound :  $\forall$  {l : List String}, nodupNames l = true  $\rightarrow$  l.Nodup
```

okB_sound

That check means what the proposition says.

```
theorem okB_sound {f : File} (h : f.okB = true) : f.WellFormed
```

Proved, but not needed for the guarantee

A further 59 results are proved in the repository and do not appear in the tree above. That is not an oversight, and in one case it is the point.

Name mangling is the interesting case. One might expect “distinct members become distinct columns” to be what makes the signature well formed. It is not how the proof goes: `gen` checks its own output for repeated names before returning it, and `gen_wellFormed` is proved by inverting that check. So well-formedness holds of *every* schema, including ones inference could never build, and it does not rest on `mangle_injective` at all. Injectivity is what makes the design sound rather than lucky — without it the generator would be relying on a runtime check to catch a collision it should never create — but the theorem above stands without it.

The rest are results that were superseded. `toSchema_paths_nodup` and its neighbours proved distinctness from the walk invariant; once `gen` began checking the same condition itself, the shorter route won. They are kept because they say something true about inference that the check does not.

Tree ²

- `gen_edges_only_on_tables` — Only a table's own module carries edges; loaders and the registry carry none.
- `parentOfElement_shorter` — Following a table to its parent strictly shortens the path, so the tree has no cycles and the walk up from any table terminates.

Inference ⁵

- `coll_le` —
- `foldl_perm` — Folding an operation that commutes in this sense over a list does not depend on the order.
- `matchesField_mono` —
- `ref_le` —
- `scalarTy_seeScalar` —

Counts ¹

- `mem_lookup_isSome` — A key in the list looks something up.

Walk ²

- `inferCorpus_ok` — Everything inference builds is kept in key order.
- `inferFinish_ok` — The final pass, which fills in absence counts, keeps it that way.

Merge ³

- `toSchema_mem` — A table of the schema came from an observation of that path.
- `toSchema_members_nodup` — No table has two columns of one name.
- `toSchema_paths_nodup` — No two tables of the schema share a path.

Order ¹

- `segOrder` — Path segments are strictly totally ordered.

Sorted ¹²

- `StrictOrder.eq_symm` — Key equality does not depend on which side is named first.
- `StrictOrder.irrefl` — Nothing is strictly below itself -- which follows from asymmetry rather than being assumed.
- `allV_mem` — A value in a list satisfies whatever all its values do.
- `combineOpt_assoc` — Nor on how three are grouped.
- `combineOpt_comm` — Combining two lookups does not depend on their order.
- `insertSorted_perm` — Inserting one entry into a sorted list permutes it.
- `lookupBy_head` — The head of a sorted list looks up to its own value.
- `mergeBy_allV` — Merging preserves a property of the values.
- `mergeBy_assoc` — Merging is associative, for combining functions with no side conditions.
- `mergeBy_comm` — Merging is commutative, proved directly by induction rather than through lookups.
- `mergeBy_right_comm` — Two merges into an accumulator commute.
- `sortBy_perm` — So insertion sort returns a permutation of its input.

Lattice ²

- `le_antisymm` —
- `le_refl` —

Mangle ³¹

- `encodeByte_append_inj` — Because it is prefix-free: one encoded byte determines where it ends.
- `encodeByte_ne_nil` — An encoded byte is never empty.
- `encodeBytes_eq_nil` — Only the empty name encodes to nothing.
- `encodeBytes_injective` — The byte encoding is injective.
- `encodeBytes_ne_f2` — No encoded name can begin with the escape prefix.
- `encodeBytes_ne_snoc_underscore` — Nor can it end in an underscore.
- `encodeBytes_ne_underscore2` — Nor can it be a lone pair of underscores.
- `hexDigit_inj` — Hex digits are distinct.
- `hexDigit_ne_underscore` — And none of them is an underscore.
- `hexVal_hexDigit` — Reading a hex digit back gives what was written.
- `isKeyword_props` — What it means to be a keyword, as the proof uses it.
- `keywordSuffixed_inj` — Distinct keywords stay distinct once escaped.
- `keywords_len` — Every OCaml keyword is at least two characters.

- `lt_128_valid` — Every byte below 128 is a valid character.
 - `mangleChars_not_keyword` —
 - `mangleChars_of_mangle` — The mangled name is the encoding of the name's characters.
 - `mangle_injective` — Two distinct member names never give one OCaml identifier, so two members never become one column and two tables never one module.
 - `mangle_not_keyword` —
 - `mangle_starts_lower` — A mangled name begins with a lowercase letter, which is what makes capitalising one character injective.
 - `no_underscore_in_keywords` — No keyword contains an underscore.
 - `pass_ne_underscore` — A character that passes through unescaped is not the escape character.
 - `passesThrough_lt` — Characters that pass through are exactly the safe ones.
 - `prefixedCore_inj` — Prefixed and unprefixed names stay disjoint.
 - `prefixedCore_ne_snoc` — A prefixed name is never a keyword with an underscore appended.
 - `prefixedCore_starts_lower` — The escaped form begins with a lowercase letter too.
 - `startsLower_cons` — Prefixing a lowercase letter keeps a name starting lowercase.
 - `toNat_inj` — Character codes are distinct for distinct characters.
 - `toNat_lt_256` — An encoded byte is a byte.
 - `toNat_ofNat` — Round-tripping a byte through a character is the identity.
 - `toNat_ofNat_pass` — The same, for characters that pass through unescaped.
 - `toUTF8_data_toList_inj` — Distinct strings have distinct UTF-8 encodings.
-

06

What is not proved

No goal in the repository is left open, so this section is about scope rather than about unfinished work.

1. **That inference never trips the generator's check.** Correctness 3–6 hold because `gen` checks its input has the shape a corpus produces and refuses one that does not. That is the stronger arrangement for a reader of the output — the properties hold of every file emitted, whatever it was handed — but it does not by itself say the check never fires on a real corpus. It does not: tables are keyed by path, a repeated member is refused, and the walk records a table before descending into what it holds. Turning that sentence into a theorem is the obvious next piece.
2. **That the emitted OCaml compiles.** Well-formedness constrains names, not references. A module naming a unit that is compiled after it satisfies the definition and still does not build. The generator does emit units in a topological order; nothing yet states that it is one.
3. **Anything about rows surviving.** This program emits a description of tables and never emits a row, so there is no object for a preservation theorem to be about. A round trip could only ever hold up to member order, explicit `null` versus an absent key, and the written form of a number — all three discarded by design.